

Gymnázium J. K. Tyla



AI asistované softwarové inženýrství

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

2026

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Práce zkoumá přechod od konverzační asistence k delegovanému agentnímu vývoji a roli harnessu jako běhového a integračního prostředí coding agenta. Cílem je vysvětlit, jak propojení jazykového modelu s nástroji, stavem, verifikací a lidskými kontrolními body umožňuje řízené provádění softwarových úloh v praxi. Agentické inženýrství práce vymezuje jako soubor postupů, které činí AI-asistovaný vývoj účinným, kontrolovaným, opakovatelným a škálovatelným. Teoretická část vychází z odborných publikací a dokumentace nástrojů. Praktická část implementuje jednoduchou produkční pipeline DarkFactory, v níž je GitHub vývojovým prostředím, GitHub Actions výpočetním prostředím, kontejner izolační vrstvou a GitHub Issues stavovým systémem. Je popsáno přijetí a plánování požadavku, lidské schválení, izolovaná implementace na větví, automatická review smyčka, zpětná vazba, merge a odstranění větve. Výsledky ukazují, že praktická autonomie vzniká především rozdělením odpovědnosti mezi model, harness, GitHub a člověka.

Klíčová slova: jazykové modely; coding agents; harness; Agentické inženýrství; GitHub Actions; DarkFactory

Abstract

This thesis examines the transition from conversational assistance to delegated agentic development and the role of the harness as the runtime and integration environment of a coding agent. Its objective is to explain how connecting a language model to tools, state, verification, and human control points enables the controlled execution of software tasks in practice. Agentic Engineering is defined as a set of practices that make AI-assisted development efficient, controlled, repeatable, and scalable. The theoretical part is based on academic publications and tool documentation. The practical part implements a simple production pipeline, DarkFactory, in which GitHub is the development environment, GitHub Actions is the execution environment, a container provides isolation, and GitHub Issues serves as the state system. It describes request intake and planning, human approval, isolated implementation on a branch, an automatic review loop, feedback, merging, and branch deletion. The findings indicate that practical autonomy arises primarily from the division of responsibility among the model, the harness, GitHub, and the human.

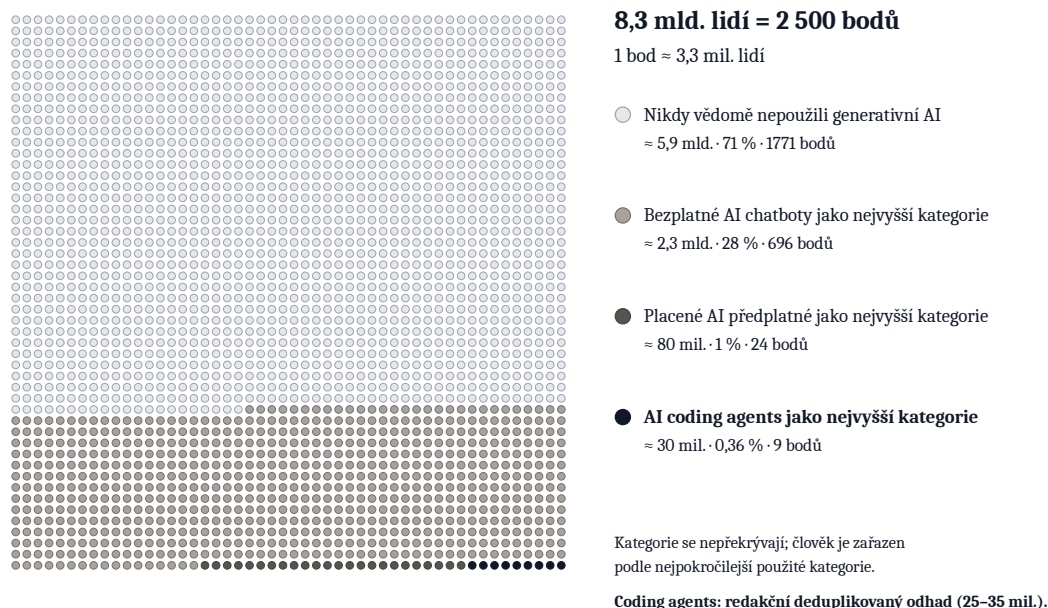
Keywords: language models; coding agents; harness; Agentic Engineering; GitHub Actions; DarkFactory

Obsah

Prohlášení	2
1 Úvod	5
1.1 Cíl a vymezení	5
2 Teoretická část	6
2.1 Jazykový model v agentním systému	6
2.1.1 Context window a kompakce	6
2.2 Agent a harness	7
2.2.1 Smyčka	7
2.3 Agentické inženýrství	8
2.3.1 Zadáání a kontext	8
2.3.2 Orchestrace a lidská integrace	8
3 Praktická část	11
3.1 Metodika	11
3.2 Architektura produkčního běhu	12
3.3 Interpretace a plánování požadavku	14
3.4 Implementace a automatická revize	15
3.5 Zpětná vazba, schválení a úklid	16
4 Výsledky a diskuse	16
4.1 Zjištění	16
4.2 Diskuse	18
5 Závěr	19
Seznam zdrojů	20
Seznam obrázků a tabulek	22

1 Úvod

Nástroje založené na velkých jazykových modelech prošly rychlým vývojem: od doplňování kódu při psaní v editoru přes konverzační chatboty až po autonomní agenty, kteří pomocí nástrojů samostatně provádějí změny a spouštějí příkazy v běhovém prostředí (1; 2; 3; 4; 5). S rostoucími schopnostmi modelů roste i jejich adopce, avšak většina uživatelů nemá představu čeho tyto nástroje jsou skutečně schopny. Většina populace se s generativní AI setkává jenom na povrchu prostřednictvím chatbotů a to bezplatným tedy omezeným přístupem. Odhad zdroje klade počet uživatelů chatbotů na 28% populace zatímco pravidelné užití AI coding agents pouze na 0,36 % světové populace(6).



Obrázek 1: Odhad rozdělení uživatelů generativní AI podle typu. Hodnota 30 milionů pravidelných uživatelů coding agents, přibližně 0,36 % světové populace, je redakční střed odhadovaného rozmezí 25–35 milionů (6). Aby mohl agent samostatně pracovat na projektu, nestačí pouhé generování odpovědí. Potřebuje kontext z repozitáře, přístup k prostředí, nástroje pro spouštění příkazů, uchování stavu mezi jednotlivými kroky a vymezený bod, v němž člověk rozhodne o přijetí výsledku (7; 8).

1.1 Cíl a vymezení

Cílem práce je ukázat, jak harness dělá z jazykového modelu autonomního agenta a jaké postupy umožňují využívat agenty účinně a kontrolovaně.

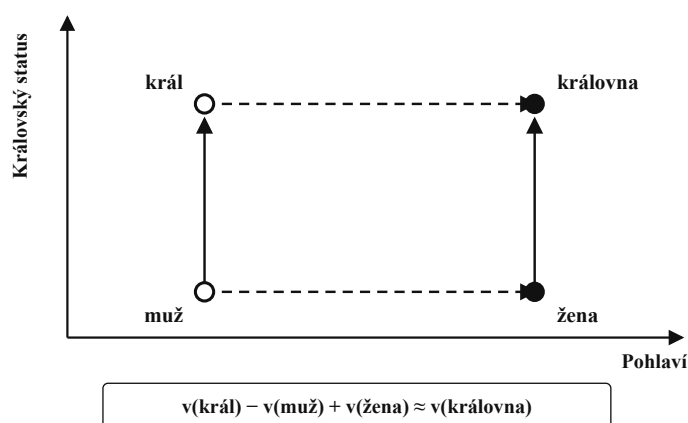
Praktická část analyzuje DarkFactory, jednoduchou produkční pipeline pro AI-asistovaný softwarový vývoj. GitHub je v ní vývojovým prostředím, GitHub Actions zajišťuje běhy, Docker kontejner odděluje prostředí, Github Issues slouží jako plánovací/stavový systém a pythonovský řadič ve spolupráci s produkčními harnessy např. Claude, agy, Codex nebo OpenCode realizuje interpretaci, plánování, implementaci, revizi a integraci změn.

2 Teoretická část

2.1 Jazykový model v agentním systému

Jazykový model (LLM) předpovídá další token na základě předešlých obsaženém v **kontextu**. **Transformer** využívá mechanismu attention (9; 10) pro zpracování vztahů mezi jednotlivými tokeny. Při inferenci model zpracuje obsah kontextového okna a vytvoří posloupnost výstupních tokenů. Samotná **Inference** však nemění soubory, nespouští příkazy ani neuchovává stav mezi kroky. Tyto činnosti zajišťuje harness, který modelu zpřístupňuje nástroje a pomocí rekursivního procesu zvaného ReAct dělá z jednotné generace souvislou konverzaci (11).

Vektorové reprezentace, označované jako **embeddingy**, zachycují sémantické vztahy ve vektorovém prostoru. Známým příkladem je vztah mezi vektory slov král, královna, muž a žena (12). Tento vztah schematicky znázorňuje Obrázek 2.



Obrázek 2: Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov **král**, **královna**, **muž** a **žena** (12).

2.1.1 Context window a kompakce

Kontextové okno (**context window**) tvoří pracovní kontext jednoho volání modelu. Může obsahovat instrukce, části repozitáře, historii volání nástrojů i výsledky předchozích kroků. Jeho kapacita však sama o sobě nezaručuje, že model všechny podstatné informace správně využije: úspěšnost jejich vybavení závisí také na umístění v kontextu a může s rostoucí délkou vstupu klesat (13). Toto postupné zhoršování práce s nahromaděným kontextem se označuje jako **context rot** (14).

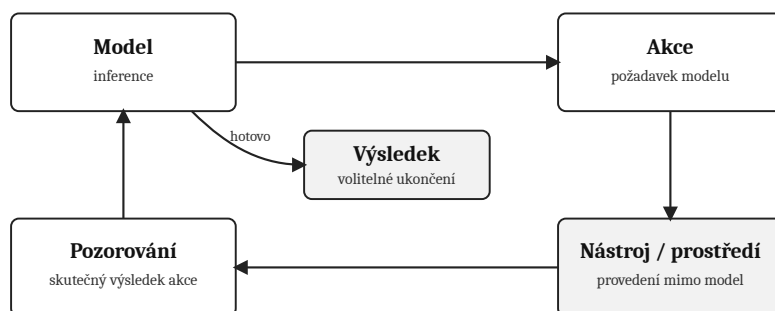
Kompakce (**compaction**) po překročení stanoveného limitu nahrazuje starší průběh strukturovaným souhrnem klíčových rozhodnutí a dosažených výsledků. Do dalšího volání tak není nutné vkládat celý přepis předchozí interakce (14).

2.2 Agent a harness

Praktickou hranicí mezi konverzačním chatbotem a agentem je míra delegovaného provádění. V běžném chatovém režimu model především vrací text a uživatel zůstává vykonavatelem navržených kroků. Agent naproti tomu prostřednictvím harnessu získává řízený přístup k nástrojům a prostředí: může procházet soubory, upravovat kód, spouštět testy a používat jejich výstupy v dalších krocích (11; 15). Role člověka se tím může posunout od provádění jednotlivých kroků k zadání, omezení a revizi delegované práce.

2.2.1 Smyčka

Základním mechanismem agentického systému je iterativní řídicí smyčka podle vzoru **ReAct (Reasoning and Acting)** (16). Model v každém kroku na základě aktuálního kontextu zvolí mezi možnostmi rozvažování, volání nástroje ve strukturovaném požadavku nebo zprávy v chatu (konečná akce). Harness každý krok přidá do přepisu konverzace a nový přepis se vrátí modelu. V případě použití nástroje tuto akci provede v běhovém prostředí, zachytí výsledek a vrátí jej modelu jako nové pozorování.



Obrázek 3: Agentní smyčka ReAct: model navrhne akci, harness ji zprostředkuje a vykoná v prostředí a pozorování se vrací do dalšího kroku(16).

2.3 Agentické inženýrství

Agentické inženýrství (**Agentic Engineering**) se snaží definovat přístup aplikovatelný pro jakékoliv AI-asistované inženýrské práce, označuje jak systematický design systému a přístupu k práci tak samotné AI-asistované inženýrství v jakémkoliv kontextu. Mezi hlavní oblasti patří: **Prompt Engineering**, **Context Engineering**, **Harness Engineering**, **Loop Engineering** a **Workflow/Graph Engineering** (7; 14; 15; 17; 18). Cílem je, aby vývojář mohl efektivně a kontrolovaně delegovat dílčí úkoly agentovi, aniž by ztratil přehled o záměru, rozsahu a kvalitě výsledku.

2.3.1 Zadání a kontext

Spolehlivé delegování práce začíná explicitním vymezením cíle, rozsahu, omezení a podmínek přijetí. Specifikace popisuje nejen požadovaný výsledek, ale také části systému, které se měnit nemají, a způsob, jakým bude výsledek ověřen. Tento přístup, označovaný jako **spec-first** nebo **spec-driven development**, dává agentovi před implementací měřitelné hranice a člověku podklad pro posouzení výsledku, je také používán v klasickém vývoji softwaru.

Prompt engineering se soustředí na formulaci instrukcí, omezení, příkladů a očekávaného výstupu konkrétního inferenčního kroku (17). **Context engineering** řeší širší a průběžný výběr, uspořádání, obnovování a kompakci informací, které má model v daném kroku k dispozici (14).

Nástroje umožňují agentovi číst a upravovat soubory, vyhledávat nebo spouštět příkazy; **Skills** spojují opakovaně použitelné instrukce, skripty a zdroje pro určitý typ úlohy (19). **Hooks** reagují na události životního cyklu a mohou před akcí či po ní vynutit deterministickou kontrolu (20). **Model Context Protocol (MCP)** standardizuje napojení externích nástrojů a datových zdrojů prostřednictvím rozhraní klient-server (21). Instrukce lze uchovat ve standardizovaném souboru `AGENTS.md` (22) přímo v repozitáři (Anthropic ojedinele využívá **CLAUDE.md**). Skilly skripty a hooky lze uchovat pod složkou `.agents/` v projektu nebo v konfigurační složce harnessu.

2.3.2 Orchestrace a lidská integrace

Pokud mají podúlohy jasné hranice a jejich výsledky lze znovu integrovat, rozsáhlou úlohu lze rozdělit do více agentních běhů. Ve vzoru **coordinator/subagent** koordinátor deleguje dílčí úkol specializovanému subagentovi s vlastním kontextem a přebírá jeho výsledek. Nezávislé podúlohy mohou zpracovat paralelní pracovníci. **Workflow graph** předem určuje závislosti, pořadí a větvení fází (18). **Agent Swarm** dynamicky rozkládá úlohu na heterogenní podproblémy a spouští specializované agenty paralelně pod řízením orchestrátoru (23).

Více agentů samo o sobě nezaručuje lepší výsledek. Paralelizace přináší užitek jen tehdy, když jsou omezeny vzájemné závislosti a koordinátor dokáže odhalit konflikty, ověřit dílčí výstupy a posoudit sloučený výsledek vůči společným podmínkám přijetí. Orchestrace proto zahrnuje nejen rozdělení práce, ale také správu kontextu, pořadí kroků, sdíleného stavu a integračních kontrol. Orchestrace je však nezbytná pro rozsáhlé úlohy, které by nebylo možné implementovat v rozsahu jednoho kontextového okna, spolehání na kompakci by vedlo ke katastrofické divergenci a kdybyse agent soustředil na celek nebyl by schopen efektivně implementovat jednotné části zadání.

Goal loop označuje nadřazenou řídicí smyčku: po dílčím dokončení ReAct smyčky harness porovná pozorovaný stav s cílem a podmínkami přijetí a podle výsledku běh ukončí, nebo zahájí další iteraci či změnu strategie. (16).

human-in-the-loop (HITL) označuje bod kde se do automatizované smyčky doplní kontrolní brány, v nichž je vyžadováno explicitní lidské rozhodnutí, například schválení specifikace, potvrzení implementačního plánu nebo přijetí výsledného diffu. Vývojář tak může ponechat agentovi ohraničenou implementační práci a současně si zachovat odpovědnost za záměr a integraci.

PRAKTICKÁ ČÁST

DarkFactory

3 Praktická část

3.1 Metodika

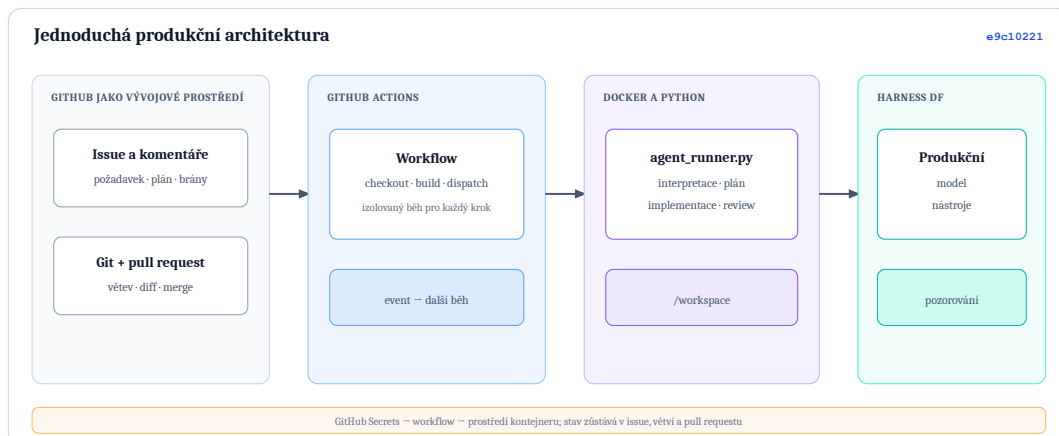
Praktická část implementuje záměrně jednoduchou produkční pipeline, v níž je vývojovým prostředím přímo GitHub a harness produkční coding agent. Cílem je ukázat, jak lze spojit události GitHubu, automatizované plánování, izolovanou práci v kontejneru a lidskou integraci do jednoho opakovatelného procesu.

Předmětem analýzy jsou čtyři navazující vrstvy:

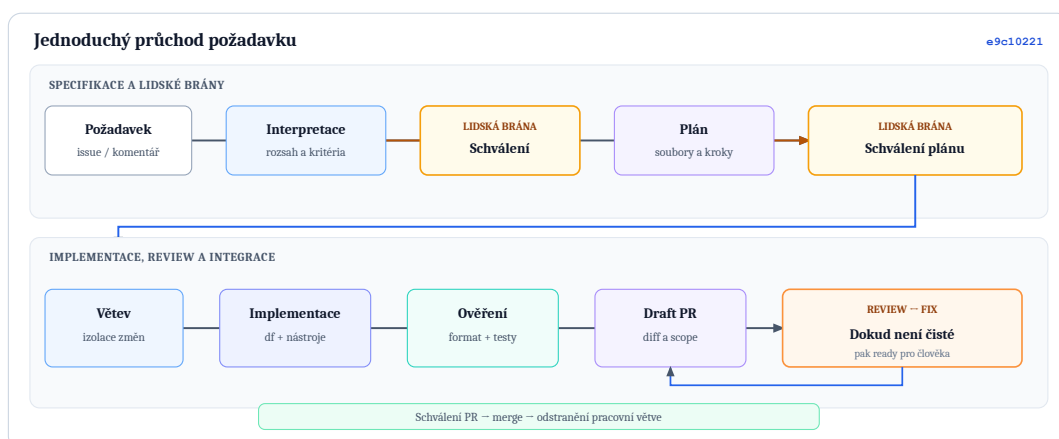
1. **GitHub jako zdroj pravdy:** issue a jeho komentáře uchovávají požadavek, schválení, plán a zpětnou vazbu; větev, commit a pull request uchovávají změnu a její průběžnou revizi.
2. **GitHub Actions jako výpočetní prostředí:** jednotlivé události spouštějí krátké workflow, která checkoutují repozitář, sestaví obraz, spustí agenta a provedou následnou integraci.
3. **Python a Docker jako izolační vrstva:** workflow předá událost a pracovní strom pythonovskému runneru v kontejneru; runner volá harness a předává mu nástroje, přihlašovací údaje a stav úlohy.
4. **Review smyčka jako podmínka integrace:** automatická revize diffu, opravy a další revize se opakují do té doby, než se uzavře poslední nález; teprve poté je pull request předán člověku.

3.2 Architektura produkčního běhu

DarkFactory nepotřebuje pro základní průchod samostatný server, databázi ani běžícího agenta na vlastním počítači. GitHub slouží jako rozhraní i jako trvalý stavový systém, každá práce agenta probíhá jako izolovaný běh v GitHub Actions.



Obrázek 4: Architektura: GitHub poskytuje události a vývojový stav, Docker odděluje běh a produkční harness agy, Claude, Codex či OpenCode nebo později vlastní df zajišťují model, nástroje a pozorování (24).



Obrázek 5: Průchod požadavku: implementace a plánování jsou odděleny lidskými bránami, implementace probíhá na větvi a review smyčka pokračuje do vyřešení nálezů (24).

Workflow `agent.yml` reaguje na otevření issue, nový komentář, review komentář, `repository_dispatch` nebo ruční spuštění. Podmínka na úrovni jobu ověřuje, zda je agent pro repozitář povolen, a filtruje automatické komentáře, aby vlastní výstup pipeline nevytvářel nové události. Po volbě cílového repozitáře workflow checkoutuje jeho pracovní kopii, sestaví obraz podle `docker/Dockerfile.agent` a spustí příkaz `dispatch` v kontejneru. Pythonový runner není náhradou harnessu. Je rozhodovací a integrační vrstvou, která převádí událost GitHubu na konkrétní agentní krok, zpracovává jeho výstup a vyvolává další událost. Modelové kroky jsou přitom stále prováděny harnessem nad explicitně předaným pracovním stromem (24).

Přihlašovací údaje se neukládají do repozitáře. GitHub App nebo jiný autorizovaný token se používá pro checkout, issue, pull requesty a push; přihlašovací údaje modelových providerů

jsou předány workflow jako GitHub Secrets a následně prostředím kontejneru. Tím pipeline odděluje své automatizační oprávnění od přihlašovacího materiálu agenta a umožňuje změnit poskytovatele bez změny pracovního stromu (24).

3.3 Interpretace a plánování požadavku

Výchozím bodem je issue s formulovaným požadavkem. Runner načte jeho titulek a text, vyžádá si od modelu interpretaci a zapíše výsledek jako komentář. Komentář má oddělit doslovné shrnutí požadavku, architektonický rozsah a návrh verifikace. Tím se z chatové odpovědi stane zkontrolovatelný návrh, který lze před dalším během přijmout nebo opravit. Po schválení interpretace je workflow spuštěno znovu. Model nyní dostane schválený požadavek a sestaví implementační plán, ve kterém uvádí očekávané změny, soubory nebo oblasti repozitáře a kroky ověření. Plán se opět zobrazí v issue. Schválení je tak explicitní bránou: samotná schopnost agenta plán vytvořit neznamená oprávnění měnit kód. Zpětná vazba člověka není součástí nového vývoje od začátku. Komentář se změnou nebo odmítnutím se předá zpět interpretaci nebo plánování, takže se opravuje rozhodnutí před vytvořením pracovní větve. Tento jednoduchý model odděluje porozumění zadání, plánování a vlastní implementaci bez potřeby složitého grafového orchestrátoru (24).

3.4 Implementace a automatická revize

Po schválení plánu runner vytvoří nebo načte pracovní větev odvozenou z výchozí větve. Implementační instrukce obsahuje schválený plán, omezení na jeho rozsah a pravidla pro testy a dokumentaci. Harness následně může procházet repozitář, upravovat soubory a používat nástroje nad `/workspace`; změny zůstávají izolované mimo výchozí větev.

Po implementaci runner spustí dostupné formátovací nástroje a deklarované testovací sady. Při neúspěchu předá výstup kontroly agentnímu kroku `fix`, který má opravit chybu bez opuštění schváleného rozsahu. Následně runner vytvoří commit, odešle větev a otevře draft pull request. Tím se oddělí samotná změna od jejího posouzení: implementace může být dokončena, ale pull request ještě není připraven k merge (24).

Na draft pull requestu začíná automatická review smyčka. Každá iterace načte aktuální větev a diff, provede deterministickou kontrolu souborů proti plánu a následně předá diff modelové revizi. Pokud review najde chyby, chybějící testy, nevhodný rozsah nebo jiný problém, runner zveřejní nález a spustí nový GitHub Actions běh s fází opravy. Oprava změní větev a spustí další review. Smyčka pokračuje, dokud review nevrátí žádný akční nález; opakovaný stejný nález bez progresu se naopak označí jako zablokovaný stav (24).

Po čisté review ještě proběhne kontrola souladu výsledného diffu se schváleným plánem. Teprve když jsou čisté obě kontroly, je draft pull request označen jako připravený k lidské revizi. Tento krok je důležitý, protože automatická revize může skončit bez nálezu, aniž by sama zaručila, že implementace odpovídá původnímu zadání.

3.5 Zpětná vazba, schválení a úklid

Po otevření pro lidskou revizi může člověk použít schvalovací workflow nebo zpětnou vazbu na pull requestu. Změnový požadavek zachycuje runner jako opravný běh na stejné větvi. Agent obdrží plán, konkrétní feedback a aktuální kontext větve, provede úpravu, commitne ji a znovu spustí automatickou review smyčku. Tím se zpětná vazba nevrací do schváleného plánu ani nevyžaduje nový počátek celého požadavku; zachovává se pouze řetězec revizí na jednom pull requestu (24).

Schválení pull requestu zpracovává samostatný workflow. Po ověření, že akci provedl autor issue nebo oprávněný člen repozitáře, převede draft na ready stav, zkontroluje požadované schválení a zapne merge. Při úspěchu se použije `--delete-branch`, takže se pracovní větev po sloučení odstraní. Integrace tedy nekončí pouhým otevřením pull requestu: zahrnuje přechod do ready, splnění ochrany větve, merge a bezpečné odstranění větve (24).

Tato architektura je záměrně jednoduchá. GitHub poskytuje události, schválení, uložení změn a průběh kontroly. GitHub Actions poskytuje výpočet. Docker odděluje běh, Python koordinuje, harness vykonává agentní práci. Složitější služby, trvalý stavový server a více souběžných agentů jsou záměrně mimo základní návrh. Výhodou je snadná reprodukovatelnost a viditelnost každého rozhodnutí. Výhodou je zároveň závislost na dostupnosti GitHubu a na kvalitě promptu, modelu a pravidel repozitáře, kterou samotná automatizace neodstraňuje.

4 Výsledky a diskuse

4.1 Zjištění

První zjištění se týká volby vývojového prostředí. Požadavek, jeho schválení, plán, zpětná vazba i výsledná revize zůstávají v GitHubu, takže jednotlivé běhy nemusí sdílet vlastní databázi ani trvalý proces. GitHub Actions přijme událost, připraví checkout a kontejner a následně spustí pythonovský runner. Runner předá práci harnessu a výsledek převede zpět do issue, větve nebo pull requestu (24).

Druhé zjištění potvrzuje oddělení lidského rozhodnutí od modelového návrhu. Issue nejprve vyvolá interpretaci, následně plán a teprve po schválení obou kroků se vytvoří pracovní větev a spustí implementace. Tím pipeline nevytváří změny v hlavní větvi pouze na základě samotného návrhu agenta (24).

Třetí zjištění se týká průběhu revize. Implementace je nejprve odevzdána jako draft pull request. Automatická kontrola nejprve porovná změněné soubory se schváleným plánem a poté předá diff modelové review. Nalezené problémy spouštějí samostatný běh opravy, po němž následuje nová kontrola. Teprve čistá review a kontrola souladu s plánem otevrou pull request pro lidskou revizi (24).

Čtvrté zjištění se týká zpětné vazby a integrace. Změnový požadavek na pull requestu spouští opravu na stejné větvi a po pushi znovu vstupuje do review smyčky. Schválení pull requestu naopak spouští merge s odstraněním větve. Pipeline tedy nekončí automatickým vytvořením kódu, ale přechází do explicitního lidského schválení a následné integrace (24).

4.2 Diskuse

Zjištění lze interpretovat jako konkrétní podobu Agentického inženýrství: praktická autonomie není vlastností samotného modelu, ale výsledkem návrhu prostředí, v němž model pracuje. GitHub poskytuje trvalý kontext a lidskou odpovědnost, Python určuje přechody mezi kroky, Docker omezuje běhové prostředí a harness propojuje model s nástroji a pozorováními. Tato dělba odpovědnosti je v souladu s principy efektivního a kontrolovaného systému popsány v teoretické části (7; 8; 24).

Review smyčka ukazuje výhodu a zároveň omezení průběžné kontroly. Deterministická scope kontrola může zachytit změnu mimo plán a testy mohou poskytnout konkrétní zpětnou vazbu, ale modelová revize zůstává pravděpodobnostní. Opakované iterace zvyšují počet příležitostí k nálezům, nikoli záruku konečné správnosti. Zablokování při opakování stejného nálezu je proto rozumnou ochranou před nekonečným během, nikoli důkazem vyřešení problému (24).

Jednoduchost návrhu má také ekonomickou a epistemickou cenu. Nevyžaduje databázi, dlouho běžícího agenta ani samostatnou orchestrátorskou službu, a celý průběh je dohledatelný v issue, commitech, kontrolách a pull requestu. Na druhé straně pipeline závisí na dostupnosti GitHubu, kvalitě issue, schopnosti modelu pracovat s nástroji a pravidlech, která definují akceptační podmínky. Jednoduchá architektura tedy usnadňuje reprodukci postupu, ale nenahrazuje odpovědnost člověka za architekturu, bezpečnost a přijetí výsledku (24).

Deterministické a modelové kontroly se proto doplňují, nenahrazují. Formátovací nástroje, testy a kontrola scope poskytují opakovatelné pozorování; modelové review, plánování a opravy interpretují zadání a neočekávané nálezy. Tato kombinace odpovídá cíli Agentického inženýrství zvýšit užitečnost delegované práce při zachování explicitních bran, ale její skutečná kvalita zůstává závislá na datech, promptu, modelu a konkrétním repozitáři (24).

5 Závěr

Jazykový model sám o sobě pouze generuje výstup v závislosti na vstupním kontextu. Agent z něj vzniká až propojením s nástroji, prostředím, stavem a pozorováním, které zajišťuje harness (7; 11). Ani toto propojení však pro účinné nasazení ve vývoji softwaru nestačí bez postupů Agentického inženýrství, které vymezují rozsah autonomie, určují body lidského rozhodnutí a zajišťují ověřování výsledků.

Implementace pipeline DarkFactory ukázala jednoduchou produkční realizaci těchto postupů. GitHub slouží jako vývojové prostředí a místo trvalého stavu, GitHub Actions jako výpočetní prostředí, Docker kontejner jako izolace a produkční harness pro modelovou práci s nástroji. Požadavek prochází interpretací a plánováním, implementace probíhá na samostatné větvi a změna je předána přes draft pull request a opakovací smyčku review–fix. Po čisté review následuje lidské schválení, merge a odstranění větve.

Úloha vývojáře se v tomto uspořádání posouvá od rutinního provádění kódu k formulaci zadání, schvalování plánu, definování akceptačních podmínek a rozhodování, zda je výsledek připraven k integraci. Práce tedy nepopisuje plnou autonomii agenta, ale kontrolované předání vybrané části softwarového inženýrství.

Zjištění zároveň ukazují meze této realizace. Modelové review může být opakované, ale není deterministickou zárukou správnosti; opakovaný nález bez progresu vede k blokaci.

Seznam zdrojů

1. GITHUB. *GitHub Copilot code suggestions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/concepts/completions/code-suggestions>
2. GITHUB. *Asking GitHub Copilot questions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/chat-with-copilot/chat-in-ide>
3. GITHUB. *Use GitHub Copilot agents* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/use-copilot-agents>
4. OPENAI. *Introducing Codex* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-codex/>
5. OPENAI. *Introducing the Codex app* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-the-codex-app/>
6. HILLEBRANDT, Finn. *How Many People Use AI? Facts & Figures 2026* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://www.gradually.ai/en/how-many-people-use-ai/>
7. ANTHROPIC. *Harness design for long-running application development* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
8. ANTHROPIC. *Scaling Managed Agents: Decoupling the brain from the hands* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/managed-agents>
9. VASWANI, Ashish, Noam SHAZEER, Niki PARMAR, Jakob USZKOREIT, Llion JONES, Aidan N. GOMEZ, Łukasz KAISER a Illia POLOSUKHIN. Attention Is All You Need. *arXiv preprint arXiv:1706.03762* [online]. 2017. Dostupné z: <https://arxiv.org/abs/1706.03762>
10. BROWN, Tom B., Benjamin MANN, Nick RYDER, Melanie SUBBIAH, Jared KAPLAN, Prafulla DHARIWAL, Arvind NEELAKANTAN, Pranav SHYAM, Girish SASTRY, Amanda ASKELL a OTHERS. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* [online]. 2020, **33**. Dostupné z: <https://arxiv.org/abs/2005.14165>
11. ANTHROPIC. *Building effective agents* [online]. 2024 [vid. 2026-09-23]. Dostupné z: <https://www.anthropic.com/engineering/building-effective-agents>
12. MIKOLOV, Tomas, Wen-tau YIH a Geoffrey ZWEIG. Linguistic Regularities in Continuous Space Word Representations. In: *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* [online]. 2013, s. 746–751. Dostupné z: <https://aclanthology.org/N13-1090/>

13. LIU, Nelson F., Kevin LIN, John HEWITT, Ashwin PARANJAPE, Michele BEVILACQUA, Fabio PETRONI a Percy LIANG. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* [online]. 2024, **12**, 157–173. Dostupné z: <https://arxiv.org/abs/2307.03172>
14. ANTHROPIC. *Effective context engineering for AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
15. OPENAI. *Sandbox agents: Concepts | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-js/guides/sandbox-agents/concepts/>
16. YAO, Shunyu, Jeffrey ZHAO, Dian YU, Nan DU, Izhak SHAFRAN, Karthik NARASIMHAN a Yuan CAO. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* [online]. 2022. Dostupné z: <https://arxiv.org/abs/2210.03629>
17. OPENAI. *Prompt engineering* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://platform.openai.com/docs/guides/prompt-engineering>
18. OPENAI. *Agent orchestration | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-21]. Dostupné z: https://openai.github.io/openai-agents-python/multi_agent/
19. AGENT SKILLS. *Agent Skills: an open format for extending AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://agentskills.io/home>
20. OPENAI. *Lifecycle | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-python/ref/lifecycle/>
21. MODEL CONTEXT PROTOCOL. *Model Context Protocol 2026-07-28 specification: server features* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://modelcontextprotocol.io/specification/2026-07-28/server>
22. AGENTIC AI FOUNDATION. *AGENTS.md* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://agents.md/>
23. KIMI TEAM. Kimi K2.5: Visual Agentic Intelligence. *arXiv preprint arXiv:2602.02276* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://arxiv.org/abs/2602.02276>
24. MARIUS, Patrik. *DarkFactory commit e9c10221* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://github.com/marius-patrik/DarkFactory/commit/e9c10221b40589512d262a0edb95f709b923150c>

Seznam obrázků a tabulek

- Obrázek 1 Odhad rozdělení uživatelů generativní AI podle typu. Hodnota 30 milionů pravidelných uživatelů coding agents, přibližně 0,36 % světové populace, je redakční střed odhadovaného rozmezí 25–35 milionů (6)..... 5
- Obrázek 2 Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov **král**, **královna**, **muž a žena** (12)..... 6
- Obrázek 3 Agentní smyčka ReAct: model navrhne akci, harness ji zprostředkuje a vykoná v prostředí a pozorování se vrací do dalšího kroku(16)..... 7
- Obrázek 4 Architektura: GitHub poskytuje události a vývojový stav, Docker odděluje běh a produkční harness `agy` `claude` `codex` či `opencode` nebo později vlastní `df` zajišťují model, nástroje a pozorování (24)..... 12
- Obrázek 5 Průchod požadavku: implementace a plánování jsou odděleny lidskými bránami, implementace probíhá na větvi a review smyčka pokračuje do vyřešení nálezů (24)..... 12